

Skribble Clone — Day-by-Day Plan

Based on your 12-phase roadmap. ~24 working days total (part-time pace, adjust freely — finishing beats rushing). Checkboxes so you can track progress day by day.

Phase 0: Planning & Setup — Day 1

- ☐ Write one-paragraph scope doc (v1 cut list: no DMs, no custom word packs, no mobile app, English only, guest-only auth)
 - ☐ Create GitHub repo (monorepo: client/ + server/)
 - ☐ Add .gitignore (node_modules/, .env, dist/, build/)
 - ☐ Write README skeleton (headers only)
 - ☐ Build wordBank.js (100–150 words, 2–3 categories)
 - ☐ Sketch 5 core screens on paper (Home, Lobby, Game-drawer, Game-guesser, Results)
 - ☐ Decide Socket.IO room strategy (one room per game code)
-

Phase 1: Backend Foundation — Days 2–4

Day 2

- ☐ npm init + install express, mongoose, socket.io, dotenv, cors, morgan, jsonwebtoken, bcrypt (+ dev: nodemon, eslint)
- ☐ Split app.js / server.js
- ☐ Connect MongoDB via config/db.js — confirm "MongoDB connected" log
- ☐ Set up .env / .env.example

Day 3

- ☐ Add core middleware (CORS scoped to CLIENT_URL, morgan, error handler, rate limiter)
- ☐ Attach Socket.IO to server, test "a user connected" log
- ☐ ☒ Checkpoint: server running + DB connected + socket handshake works

Day 4

- ☐ Build guest-session auth (POST /api/auth/guest → JWT with guestId + name)
 - ☐ Build socketAuth middleware (verify JWT in io.use())
 - ☐ Build Room, Game (+ embedded Round) models
-

Phase 2: Frontend Foundation — Days 5–6

Day 5

- ☐ Scaffold Vite + React
- ☐ Install deps + configure Tailwind
- ☐ Set up router.jsx (stub all 5 routes with just headings)
- ☐ Build SocketContext + useSocket hook
- ☐ ☒ Checkpoint: blank page connects/disconnects, logs socket id

Day 6

- ☐ Build AuthContext (guest name + JWT, persisted to localStorage)
 - ☐ Build GameContext (players[], currentDrawer, currentRound, scores, timeRemaining, myRole)
 - ☐ Build api.js axios instance with JWT interceptor
-

Phase 3: Rooms & Lobby System — Days 7–9

Day 7

- ☐ Build POST /api/rooms (REST, generates room code)
- ☐ Build GET /api/rooms/:code (REST, validates room)
- ☐ Build RoomManager.js (in-memory)

Day 8

- ☐ Build join-room socket event (verify JWT, add to Room doc + RoomManager, broadcast player list)
- ☐ Build leave-room / disconnect handling (reassign host if needed)

Day 9

- ☐ Build HomePage forms (Create Room, Join Room)
 - ☐ Build LobbyPage (room code + copy button, live player list, host-only Start button)
 - ☐ Handle edge cases (room full, not found, already started)
 - ☐ ☒ Checkpoint: 2 tabs join same room, see each other
-

Phase 4: Real-Time Drawing Canvas — Days 10–13

Day 10

- ☐ Build local-only DrawingCanvas.jsx (mouse events, no sockets yet)
- ☐ Capture stroke points ({x, y, color, brushSize, type})
- ☐ Add touch event support

Day 11

- ☐ Wire stroke-start / stroke-move / stroke-end socket events
- ☐ Throttle stroke-move emissions (~20–30ms)

Day 12

- ☐ Replay incoming strokes on other clients (reuse render function)
- ☐ Build Toolbar.jsx (color, brush size, clear button)
- ☐ Wire clear-canvas as broadcast event

Day 13

- ☐ Lock canvas interaction based on myRole
 - ☐ Test with 2+ real tabs/devices (including phone on same wifi)
 - ☐ ☒ Checkpoint: drawing sync feels instant and reliable in isolation
-

Phase 5: Game Logic — Days 14–17

Day 14

- ☐ Build GameManager.js in-memory state
- ☐ Build turn rotation logic (handle player leaving mid-rotation)

Day 15

- ☐ Build word selection flow (3 random words → drawer only → select-word event)
- ☐ Build server-side round timer (timestamp-based, server-authoritative)

Day 16

- ☐ Build round-start broadcast (masked word to guessers, full word to drawer)
- ☐ Build round-end logic (timer expiry OR all-correct)
- ☐ Build next-round transition

Day 17

- ☐ Build WordSelectModal.jsx (auto-pick fallback)
 - ☐ Build WordDisplay.jsx + RoundTimer.jsx (resync from server timestamp)
 - ☐ Build role-based UI switching (drawer vs guesser)
-

Phase 6: Chat & Guessing System — Days 18–19

Day 18

- ☐ Build send-guess socket event (server-side case-insensitive check)
- ☐ Handle correct guess (mark guessed, broadcast system message, block re-submission)
- ☐ Handle incorrect guess (broadcast as normal chat)
- ☐ Block drawer from guessing (server-side check)

Day 19

- ☐ Build ChatPanel.jsx + ChatMessage.jsx (3 message types)
 - ☐ Wire auto-end-round when everyone's guessed
-

Phase 7: Scoring & Leaderboard — Day 20

- ☐ Build scoring.js (time-based formula)
 - ☐ Award drawer bonus points
 - ☐ Update in-memory scores + broadcast score-update
 - ☐ Build ScoreBoard.jsx (live sidebar)
 - ☐ Persist final Game doc to MongoDB on game end
 - ☐ Build ResultsPage.jsx (standings, Play Again, Back to Home)
-

Phase 8: Polish, UX & Sound — Days 21–22

Day 21

- ☐ Add sound effects (correct guess, round-end, timer tick) + mute toggle
- ☐ Add transitions between lobby → game → results

- ☐ Add loading/connecting states

Day 22

- ☐ Verify mobile/touch support on a real phone
 - ☐ Responsive layout pass (canvas + sidebar stacking)
 - ☐ Handle empty/edge states explicitly
 - ☐ Add per-player avatar/color assignment
-

Phase 9: Testing & Bug Fixing — Days 23–24

Day 23

- ☐ Full end-to-end manual test, 3+ tabs/devices, log every bug found
- ☐ Test disconnects mid-round (host, drawer, guesser)
- ☐ Test simultaneous correct guesses (race condition check)

Day 24

- ☐ Write backend unit tests (scoring.js, maskWord.js) with Jest
 - ☐ Test under throttled network (Chrome DevTools Slow 3G)
 - ☐ Check for memory leaks in RoomManager/GameManager
-

Phase 10: Deployment — Day 25

- ☐ Deploy backend to Render (Web Service, not Static Site)
 - ☐ Deploy frontend to Vercel (set VITE_API_URL, VITE_SOCKET_URL)
 - ☐ Configure CORS to exact Vercel domain
 - ☐ Confirm Socket.IO connects over WSS in production
 - ☐ Set up MongoDB Atlas production DB + IP allowlist
 - ☐ Full live playtest with real friends on different networks/devices
-

Phase 11: Portfolio & Resume Integration — Day 26

- ☐ Write strong README (stack, architecture diagram, setup, demo link at top)
 - ☐ Add project to sanam-rai.com.np with screenshots + gameplay GIF
 - ☐ Record short gameplay GIF/Loom video
 - ☐ Add impact-style bullet to CV (lead with server-authoritative timing / stroke-sync throttling)
 - ☐ Prepare 30-second interview explanation
-

Total: ~26 working days. This is a guide, not a deadline — some days (Phase 4, Phase 9) are genuinely harder and may spill into the next day. That's normal, not falling behind.